

Şaqr at AISA-ArabFC: What Actually Influences Arabic Function Calling?

Toka Khaled¹

Al-Azhar University
Tokakhaled98@gmail.com

Eman Elrefai²

Alexandria University
eman.lotfy.elrefai@gmail.com

Ahmed Soliman³

University of Florida
ahmed.soliman@ufl.edu

Abstract

We describe Şaqr, ranked first on both scored tracks of the AISA-ArabFC shared task on Arabic function calling, scoring 0.9279 on Track A and 0.9400 on Track B over a 1,125-instance blind test set. The system is deliberately plain: one Qwen3-4B model with LoRA, two epochs, provided data only. Our contribution is not the system but an accounting of where score comes from. We find that scale helped only up to a point in our single-run, cross-family comparison, and that the remaining gains came instead from inference-time decisions worth +0.0063 Overall A and +0.0223 Overall B (Appendix A) with no retraining, prefilling the reasoning trace so the model cannot skip it, and parsing arguments against the declared schema. We also report two properties of the data that bound any system’s score: abstention is decidable from prompt structure without reading the query, and the gold annotation is self-inconsistent on precisely the slots that separate strong systems.

1 Introduction

Tool calling: mapping a request onto a structured function invocation, is the interface through which language models act rather than merely answer. Reasoning and planning pay off only once they become a correct call to the right tool with the right arguments (Patil et al., 2025). It is also the standard fix for hallucination and arithmetic error, offloading those to a database or calculator (Zhuang et al., 2023). For Arabic it is not yet routine: the best model scores 57% in English but only ~36% in Arabic on a 52-language benchmark (Kulkarni et al., 2025). The gap is not just training data, multilingual models still produce malformed Arabic calls, and English-learned skill transfers poorly (Ersoy et al., 2025). Systems must also handle dialectal/orthographic variation and arguments that may stay Arabic or canonicalise to English/ISO codes. AISA-ArabFC (Nacar et al., 2026a) is the first shared task testing this across MSA and four dialects over 27 tools, extending (Nacar et al., 2026b) within (Nacar et al., 2026c).

It was sufficient to keep our winning system simple¹: one open 4B model, LoRA on the provided split, no chat-template re-rendering, no ensembling, no unmeasured post-processing, each component beats its own absence before entering the pipeline. We rank first on Track A and Track B, third on the Track C diagnostic. Two findings:

1. **Scale helps, then stops.** In single, uncontrolled runs across model families, Overall A rises from Gemma-3 (270M, 1B) to Gemma-4 E2B and Qwen3-4B, then drops at Gemma-4 E4B (§5.2). We read this as a trend, not a controlled scaling law: the E4B point is one run and may be unfavourable (Limitations).
2. **Inference-time decisions were the last lever.** `<think>` prefill plus schema-typed parsing add +0.0063 Overall A and +0.0223 Overall B (§3.3).

2 Background

2.1 Task Setup

Given an Arabic dialect (MSA or Gulf/Egyptian/Levantine/Maghrebi) and four candidate schemas from a 27-tool registry (20 gold, seven distractor-only), a system outputs the correct function with typed key-value arguments, or the literal none. E.g., ابغى احجز عند دكتور اسنان يوم الاحد الساعة خمسة العصر maps to `book_doctor_appointment(specialty="Dentistry", day="Sunday", time="17:00")`. Two scored tracks: **Track A** (call+args, Eq. (1)) and **Track B** (call+args plus an Arabic `<think>` trace, Eq. (2)); see Appendix A. **Track C** (Cross-Dialect Robustness) is an automatic diagnostic, not a submission: per-dialect FnAcc/ArgEM with the max–min gap across dialects.

2.2 Related Work

English-centric benchmarks. ToolQA (Zhuang et al., 2023), StableToolBench (Guo et al., 2024),

¹Code: <https://github.com/tokakhaled/AISA-ArabFC>

and BFCL (Patil et al., 2025) are standard but English-only; BFCL-V3 audits show 71.5% of entries contain schema/trajectory errors (Chen et al., 2026), and GRANITE-FUNCTIONCALLING (Abdelaziz et al., 2024) likewise. Under paraphrase and toolkit expansion, SOTA models degrade sharply, 70–90% of errors from argument-value mismatches (Rabinovich and Anaby-Tavor, 2025) — motivating our schema-typed post-processing and dev-grader checkpoint selection.

Multilingual and Arabic tool calling. MASSIVE-Agents (Kulkarni et al., 2025) covers 52 languages but shows a severe low-resource cliff. Ersoy et al. (2025) translate Glaive/xLAM for Fanar; native data beats translated for argument accuracy. Nacar et al. (2026b) introduce AISA-AR-FunctionCall, underlying AISA-ArabicFC, cutting parse failures from 87% to under 1% via full fine-tuning of a 270M FunctionGemma. We instead LoRA-adapt Qwen3-4B and report a call-only vs. reasoning-primed comparison not previously reported for Arabic function calling.

3 System Overview

Our system is a single pipeline: unsloth/Qwen3-4B-Instruct-2507 (Qwen Team, 2025), LoRA-fine-tuned directly on the shared task’s AISA-ArabicFC² (Nacar et al., 2026b). This model was selected empirically: we fine-tuned and evaluated a sequence of candidate base models on dev before settling on it (Table 2). We use no additional labelled data; the only external resources are the base models’ pretrained weights, the Unsloth kernels, and the official grader (§4). Figure 1 shows the pipeline.

3.1 Training Phase

We trained and scored several base models on dev (Table 2) — and kept the one with the highest Overall A. Qwen3-4B was the final choice, and the checkpoint sweep below (§3.2) operates only on it.

Training uses the dataset’s own DSL text field verbatim (tool declarations, user turn, `<think>+call` target) — calls use a bespoke serialisation, `<start_function_call>call:name{k:<escape>v<escape>}<end_function_call>`, with string arguments escape-wrapped and numeric arguments bare — only stripping a leading `<bos>` — no chat-template re-rendering. Since

Gemma-style markers are plain multi-token text to the Qwen3 transformers-based (Wolf et al., 2020) tokenizer, we mask the SFT loss by locating the last token-id span of the response marker rather than by string search, keeping only the `<think>+call` span in the loss.

3.2 Checkpoint selection

Once Qwen3-4B was fixed as the base model, every saved checkpoint of its fine-tune is scored raw (no pre-/post-processing) on dev with the official grader; the checkpoint with the highest resulting Overall A is carried into test inference. This isolates checkpoint quality from anything the inference pipeline could add.

3.3 Inference Phase

Only after the checkpoint is frozen do we layer on three inference-time components. Their combined effect on dev is +0.0063 Overall A and +0.0223 Overall B (Table 2, last two Qwen3-4B rows).

Think priming. We append `\n<think>\n` to each prompt so the model’s first generated token continues an already-open reasoning block rather than deciding whether to open one. Without this, the model occasionally skips reasoning entirely, dropping ThinkRate below 1.0; with it, ThinkRate reaches 1.0000 on both dev and test. Since Track B weights ThinkRate at 0.20 (Eq. (2)), this single change accounts for the majority of the Overall B gain.

Schema-typed argument parsing. A naive parser that applies `int()` when no decimal point is present destroys leading zeros on identifier fields: gold 00112233 becomes 112233 and can never match. Our parser instead reads each tool’s declared parameter types from the schema at inference time and casts accordingly: fields declared as `STRING`, along with a fixed set of twelve identifier fields³, are kept as strings verbatim. `NUMBER` and `INTEGER` fields are cast via `float()` after translating any Arabic-Indic digits (٠–٩) to Western digits. Untyped values fall back to the naive heuristic. On the blind test set, this parser altered 191 of 1,125 argument sets relative to the naive baseline, the vast majority being identifier fields whose leading zeros were preserved.

²<https://huggingface.co/datasets/TuwaiqAcademy/AISA-ArabicFC>

³account_number, iban, id_number, insurance_number, iqama_number, national_id, passport_number, phone, phone_number, recipient_iban, reference_number, visa_number.

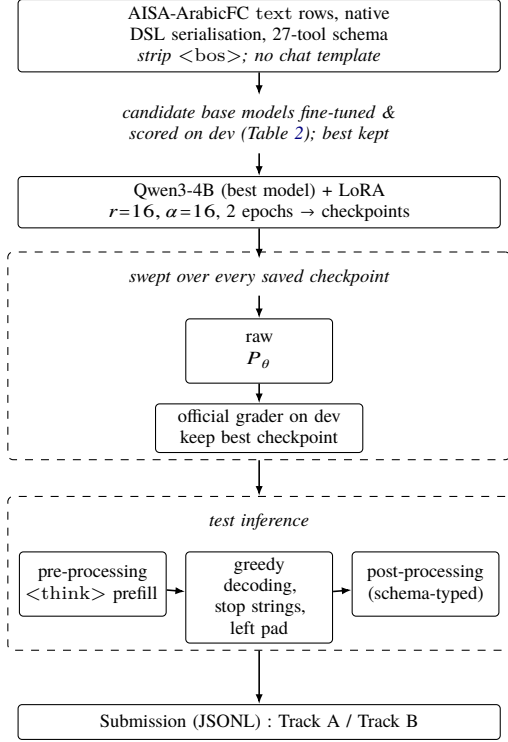


Figure 1: Pipeline.

Greedy decoding with structural stops. Generation uses greedy decoding (`do_sample=False`), a cap of 300 new tokens, and stop strings on `<end_function_call>` and `<end_of_turn>`. Padding and truncation are both set to left: right-truncation would cut the trailing `<start_of_turn>` model marker that separates prompt from generation.

4 Experimental Setup

4.1 Data Preparation

All three splits share the same preprocessing — only the leading `<bos>` literal stripped, no chat-template re-rendering (§3.1) — but are consumed differently: training tokenizes and masks the full text field, while dev/test generation uses only the prompt prefix up to and including `<start_of_turn>` model (tool declarations + user turn), obtained by splitting each row’s text on that marker. Sequences truncate at `max_seq_length = 2048` (max observed on dev: 944 tokens). Test is blind and scored once.

4.2 Training Configuration

We used base model unsloth/Qwen3-4B-Instruct-2507, LoRA ($r=16$, $\alpha=16$, dropout 0) on the attention and MLP projections, 2 epochs (1,320 steps) at effective batch size 16, learning

rate $2e-4$ with a cosine schedule and 20 warm-up steps, AdamW-8bit optimizer, and hardware acceleration through Google Colab with A100 GPU via Unsloth⁴ (Unsloth AI, 2024). Track A/B share one checkpoint and generation, differing only in whether think is included in the JSONL record.

4.3 Evaluation Metrics

Official metrics: FnAcc, ArgEM (grader-normalized names/dates/currencies/numerals/lists), ThinkRate, Overall A/B, per-dialect FnAcc/ArgEM, and accuracy gaps (`gap_fnacc`, `gap_argem`). Grader fetched at run time from the shared task’s HF Space⁵ and called unmodified; implementation details in the grader itself, not reproduced here for space.

5 Results

5.1 Official results

Our system was evaluated on both development and test sets, achieving the following results:

Split	O_A	O_B	FnAcc	ArgEM	Think	Macro _{dial}
Dev	0.9316	0.9430	1.0000	0.8860	1.0000	0.9201
Test	0.9279	0.9400	0.9973	0.8816	1.0000	0.8994

Table 1: Dev vs. official blind test. O_A/O_B =Overall A/B; Macro_{dial}=macro-avg. per-dialect Overall.

We rank **1st on Track A**, **1st on Track B**, and **3rd on the Track C**. so with ThinkRate and FnAcc pinned at 1.0000, **ArgEM is the only lever left**: one more correct row is worth $0.6/n_+ \approx 0.0006$ Overall A.

Test FnAcc (0.9973) is a hair below dev’s 1.0000 : a handful of function-selection misses appear only on the blind set, while ArgEM held nearly flat ($\Delta=-0.0044$), so the checkpoint selected on dev generalised with minimal loss.

5.2 Ablations

Scale helps until it does not. Table 2 shows the full trajectory. Overall A rises monotonically across Gemma3 270M $\rightarrow$ 1B $\rightarrow$ Gemma4 E2B $\rightarrow$ Qwen3-4B. The exception is instructive: Gemma4 E4B, the largest model we trained, scored below E2B at roughly twice the compute. This

⁴<https://github.com/unslothai/unsloth>

⁵<https://huggingface.co/spaces/TuwaiqAcademy/AISA-ArabicFC-SharedTask-Leaderboard>

System	A	B	Split
Organisers' reference model	0.7170	0.7390	dev
+ post-processing	0.7649	0.7828	dev
Gemma3 270M, 1 epoch	0.7818	0.8154	dev
Gemma3 270M, 2 epochs	0.8019	0.8180	dev
Gemma3 270M, 3 epochs	0.7951	0.8215	dev
2 epochs + post-processing	0.8019	0.8180	dev
Gemma3 1B, 2 epochs	0.8721	0.8931	dev
Gemma4 E2B	0.9112	0.9165	dev
Gemma4 E4B	0.8880*	—	dev
Qwen3-4B	0.9253	0.9207	dev
+inf. pre / post-processing	0.9316	0.9430	dev
Submitted	0.9279	0.9400	test

Table 2: Development trajectory and submitted test result. *E4B was scored on track A only

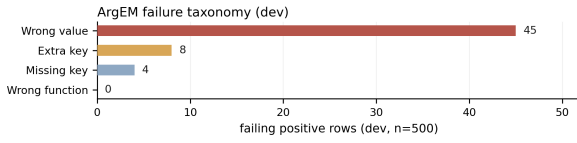


Figure 2: ArgEM failure taxonomy, dev split.

can be consistent with the ceiling being set by data quality rather than capacity.

Model chosen but the gains don’t stop. The inference pre / post-processing (§3.3) move Qwen3-4B from raw 0.9253/0.9207 to 0.9316/0.9430 Overall A/B on dev +0.0063/+0.0223 improvement with no retraining.

5.3 Error Analysis (dev split)

FnAcc is 1.0000 on all 545 dev rows in every dialect, so all remaining error lives in `args_match`: of 500 positive rows, 57 fail ArgEM, split **wrong_value** 45 (79%), **extra_key** 8 (14%), **missing_key** 4 (7%), **wrong_function** 0 (Figure 2). Errors concentrate in three tools with free-text, boundary-sensitive arguments: `compare_prices` (42%), `calculate_customs` (38%), `order_food` (33%) while the fourteen tools whose sole argument is a verbatim ID or city are error-free. Two failure modes recur: **span-boundary** disagreement, where gold itself is inconsistent (“قارن لي” “مصر والسعودية” takes `country=مصر`, while a near-identical query elsewhere takes both countries), and **list-serialisation** drift, e.g. gold شيش طاووق, predicted شيش طاووق وكوكاكولا for `order_food.items` — model and annotation disagree whether “and” or a comma delimits a list, and gold uses both conventions elsewhere. These read as un-normalised targets more

than model failures. canonicalising training targets is the most promising next step (§6).

Abstention is structurally signalled. A no-call instance is identifiable directly from the text column: it contains no `<start_function_declaration>` marker, while every positive renders four. The classes separate perfectly on the labelled splits (train 50 negatives vs. 10,500 positives; dev 45 vs. 500), and the blind test partitions the same way (125 of 1,125 rows carry no declaration). Abstention is therefore decidable from prompt structure without reading the query.

5.4 Cross-Dialect Robustness (Track C)

We post the best MSA, Egyptian, and Levantine score of the top three teams, but the widest spread (gap=0.136 vs. 0.077/0.093 for 1st/2nd), driven by Maghrebi (0.806), is why we place 3rd despite leading on three of five dialects. Maghrebi is just 1.3% of the corpus and the weakest dialect for all, pointing to data scarcity rather than a pipeline-specific flaw; on dev this appears as one miss in six rows, consistent with sample-size noise.

5.5 The annotation ceiling

Part of the residual error is unrecoverable. Three development queries state that employment ended by contract expiry: (38), وسبب الفصل إنهاء عقد (63), and النوع إنهاء عقد (243). Yet gold records `termination_type=end_of_contract` for the first and omits the field for the other two; across the 23 instances of that tool it is present in 15. The weather tool’s `days=1` default is mandated in the prompt but appears in only four of nine development instances. There’s no single choice that gets both right. We also observed instances labelled MSA whose queries are plainly Egyptian.

The top two teams have identical FnAcc (0.9973) and ThinkRate (1.0000), so their entire margin is ArgEM: 0.0006 Overall A and 0.0005 Overall B, the weight of a single argument row ($0.6/n_+$ and $0.5/n_+$ with $n_+ = 997$). These argument slots are exactly the inconsistent-gold tools of §5.3, so the gold annotation is self-inconsistent on the very slots that separate the top systems.

6 Conclusion

We presented Şaqr, a deliberately plain system for Arabic function calling: a single LoRA-adapted

Qwen3-4B trained for two epochs on the provided data only. It ranks 1st on Tracks A and B, and 3rd on Track C. Our accounting of where the score comes from yields three points: scale helps only up to a point (§5.2); the final gains came from inference-time pre- and post-processing, with no retraining; and, with FnAcc and ThinkRate saturated, the residual error is entirely argument-level, confined to tools whose gold annotation is itself inconsistent which bounds any system. Future work takes three directions. First, a larger adapter: $r=32$ rather than the submitted $r=16$, which we could not afford to evaluate. Second, ensembling several checkpoints or base models, which we excluded by design (§5.3). Third, targeted data augmentation for Maghrebi, our weakest dialect and just 1.3% of the corpus, generated for every tool. (§5.4).

Limitations

All training ran on a single Google Colab A100 under a constrained GPU budget, which fixed our search space more than our findings did: one LoRA configuration ($r=16$, $\alpha=16$), two epochs, and a single run per base model, with no seed replication and no hyper-parameter sweep. Differences between nearby configurations are therefore single-run point estimates — the E4B result in particular may reflect an unfavourable run rather than a property of the model. With more compute we would train longer, sweep the adapter settings, and repeat each configuration across seeds. Our error analysis is likewise confined to the 545-row development split, since test labels are withheld.

Acknowledgments

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task.

References

- Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, and Others. 2024. [Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*.
- Jiaao Chen, Jingyuan Qi, Mingye Gao, Wei-Chen Wang, Hanrui Wang, and Di Jin. 2026. Eigen-data: A self-evolving multi-agent platform for function-calling data synthesis, auditing, and repair. *arXiv preprint arXiv:2603.05553*.
- Asim Ersoy, Enes Altinisik, Husrev Taha Sencar, and Kareem Darwish. 2025. [Tool calling for Arabic LLMs: Data strategies and instruction tuning](#). In *Proceedings of the Third Arabic Natural Language Processing Conference*.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. *arXiv preprint arXiv:2403.07714*.
- Mayank Kulkarni, Vittorio Mazzia, Judith Gaspers, Christopher Hench, and Jack FitzGerald. 2025. [MASSIVE-agents: A benchmark for multilingual function-calling in 52 languages](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*.
- Omer Nacar, Mohammed Al Khalifa, and Saeed Alzahrani. 2026a. AISA-ArabicFC: The first shared task on Arabic function calling for agentic ai systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.
- Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem Alotaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.
- Omer Nacar, A. Deema, and A. Mohammed. 2026c. [AISA: A unified architecture for agentic ai systems](#).
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. [The Berkeley function calling leaderboard \(BFCL\): From tool use to agentic evaluation of large language models](#). In *Proceedings of the 42nd International Conference on Machine Learning*.
- Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Ella Rabinovich and Ateret Anaby-Tavor. 2025. [On the robustness of agentic function calling](#). In *Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP)*.

Unsloth AI. 2024. Unsloth: Fast and memory-efficient LLM finetuning. <https://github.com/unslothai/unsloth>.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. Toolqa: A dataset for llm question answering with external tools. *arXiv preprint arXiv:2306.13304*.

A Track Scoring Formulas

$$A = 0.40 \text{ FnAcc} + 0.60 \text{ ArgEM} \quad (1)$$

$$B = 0.30 \text{ FnAcc} + 0.50 \text{ ArgEM} + 0.20 \text{ ThinkRate} \quad (2)$$

B Additional tables

Dialect	Dev		Test	
	<i>n</i>	%	<i>n</i>	%
MSA	323	59.3	825	73.3
Egyptian	66	12.1	104	9.2
Levantine	75	13.8	102	9.1
Gulf	75	13.8	83	7.4
Maghrebi	6	1.1	11	1.0
Total	545		1,125	

Table 3: Dialect composition, counted from the released data.

C Additional Figures

D Reproduction details

Dataset We use **AISA-ArabFC**⁶ (Nacar et al., 2026b), Apache 2.0: MSA plus four dialects,

⁶<https://huggingface.co/datasets/TuwaiqAcademy/AISA-ArabFC>

#	Team	Overall	FnAcc	ArgEM	Think
<i>Track A</i>					
1	Şaqr	0.9279	0.9973	0.8816	—
2	Yalla	0.9273	0.9973	0.8806	—
3	QMUL_AK	0.9253	0.9938	0.8796	—
<i>Track B</i>					
1	Şaqr	0.9400	0.9973	0.8816	1.0000
2	Yalla	0.9395	0.9973	0.8806	1.0000
3	QMUL_AK	0.9380	0.9938	0.8796	1.0000

Table 4: Official leaderboard, blind test set, top three per track.

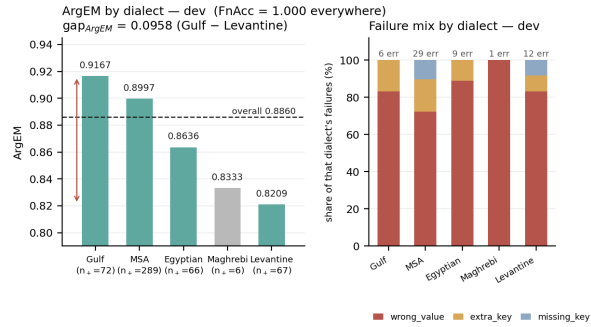


Figure 3: Task corpus dialect composition. Maghrebi is 1.3% of the corpus, supporting the data-scarcity explanation for its gap in §5.4.

eight expert-authored domains (healthcare, banking, government, Islamic services, travel, weather, e-commerce, utilities); 10,550 train (10,500/50), 545 dev (500/45), blind test, over 27 tools (20 gold, seven distractor) — comparable in scale to **Ara-bic_Function_Calling**⁷ (50,810 samples, 8 domains, 5 dialects), though no formal relation is documented.

Hyperparameters. LoRA $r=16$, $\alpha=16$, dropout 0.0, no bias, attention and MLP modules of the language model only, seed 3407. Two epochs (1,320 steps), learning rate 2×10^{-4} with cosine schedule and 20 warmup steps, adamw_8bit, weight decay 0.01, per-device batch 4 with gradient accumulation 4 (effective 16; 2×8 on the fp16 fallback path), max sequence length 2048, 33M trainable parameters (0.81%). Inference: greedy, batch 8, 300 new tokens, stop strings on the call and turn markers, padding and truncation side both left.

Preprocessing and masking. Strip the literal leading `<bos>` from each text field; split on the model-turn marker to form the prompt and append `"\n<think>\n"` at inference. Tokenise the full instance, locate the final occurrence of the marker's

⁷https://huggingface.co/datasets/HeshamHaroon/Arabic_Function_Calling

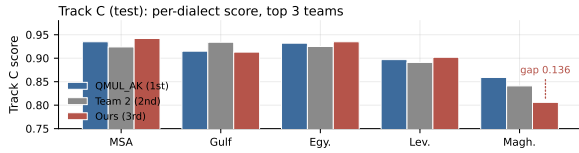


Figure 4: Track C (test), per-dialect score for the top 3 teams.(§5.4).

token-id sequence, and set labels to the ignore index up to and including it; assert no instance yields an empty answer span. No chat template is applied anywhere.

Gold value conventions. Verified without exception over all 10,500 positive training instances: schema `STRING` yields a string (16,514 values); `NUMBER` (2,983) and `INTEGER` (1,313) yield floats retaining the fractional zero. No gold value is ever an array. Arabic-Indic digits occur only inside string-typed values, including dates and identifiers, never in numeric fields.

Identifier fields kept as strings by our parser and compared as strings by the scorer: `account_number`, `iban`, `id_number`, `insurance_number`, `iqama_number`, `national_id`, `passport_number`, `phone`, `phone_number`, `recipient_iban`, `reference_number`, `visa_number`. The scorer additionally strips fourteen tool-scoped optional parameters from both sides before comparison; all remaining keys are compared under strict set equality.